Next JS

It is react framework to build full stack web applications

React Server Components (RSC) –

React comp into two categories i.e. Server comp and Client comp
Server comp – to perform server-side operations like fetching data. By default, next js treats all comp as server side

Client side – They handle user interactions and can use hooks provided by react. To created client-side comp, “use client” directive to be used at the top of the file

For Routing –
Next js uses file-based routing system in which the name of the folder is typed in URL in order to access the content
All the pages to show should have file named with **page.tsx or page.js** extension

For Eg –

```
app
 |
 |----home
 |        |-----page.tsx
 |
 |----page.tsx
```

To access root URL -> http://localhost:3000/
To access home URL -> http://localhost:3000/home

Nested Routing -

```
app
 |
 |----home
 |        |-----banner
 |                  |------page.tsx
 |
 |----page.tsx
```

To access banner – http://localhost:3000/home/banner

Note -

All routes should be placed inside app folder

Dynamic Routes –

In the condition of 100 of routes to a specific page, we use dynamic routes. It creates a route for N number of routes based on certain condition like productid

The folder should be named as [foldername]. For eg – [productID]

To access productid -> http://localhost:3000/product/productID

Catch All the Segments Route –

It renders the pages which has name of parent in url even if it is not mapped. So, any url that has its parent in url , it renders the parent if the route is not found

The folder should be named as [...foldername], For eg – [...slug]

Note-

*To render not found page, create a file with **not-found.js** or **not-found.tsx***

To invoke not found page programmatically – use notFound from "next/Navigation"

Private Folders -

Excluded from route. Used to store the ui components.
Folder should be named as => _folderName

Route Groups -
It is used to club different routes to one category. For e.g. - Forgot password, Authentication in Login grp
Naming Convention – Always start the name with small letters.

To implement, folder should be named as - (folder_name) => (auth)
Layout -
It is a file that controls how ui elemets are placed. Layout wraps all routes, so each route's component becomes {children}.

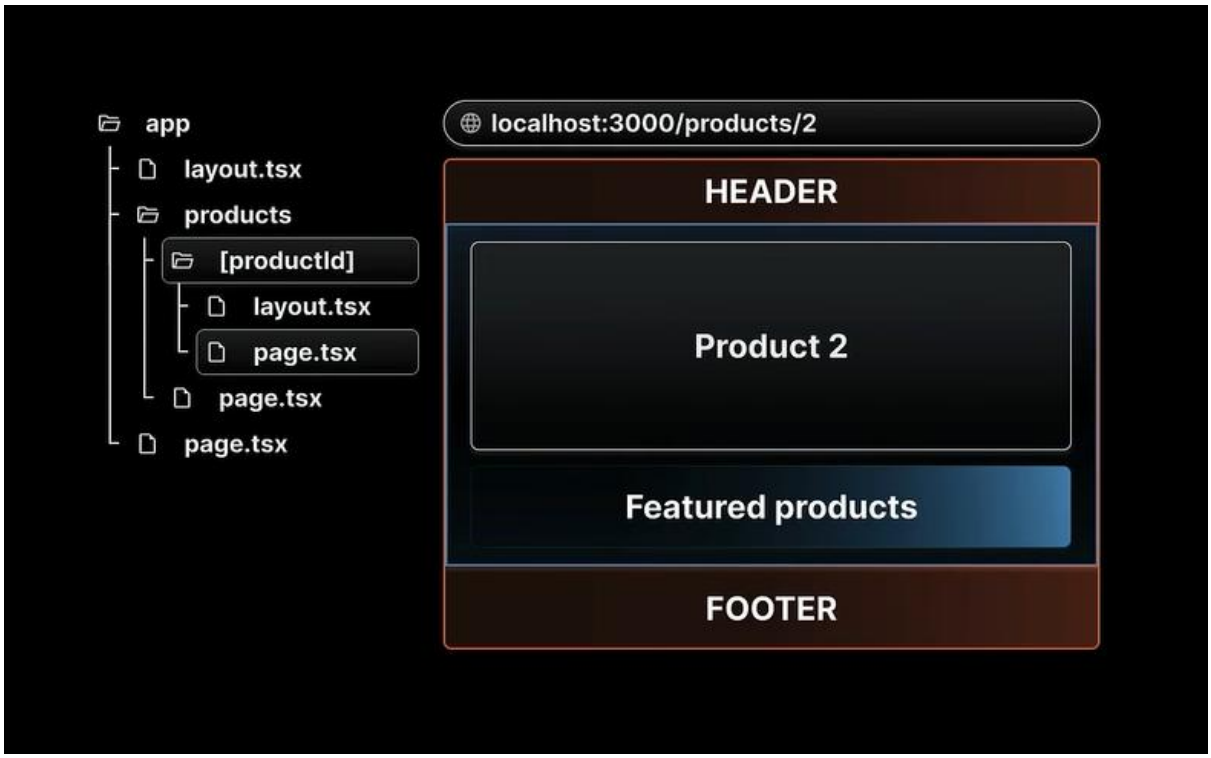
Note -

File named with layout, should have an exported function named RootLayout for normal folders
File named with layout, should have an exported function named SortedGrpsFolderName + Layout for sorted groups folder. For eg => AuthLayout etc

---

For eg – In layout.tsx file, header and footer tag enclose the children (other routes). So, accessing any of the routes, header and footer will be shown along with content

Nested Layouts -
To have a specific layout for a specific route, we create a **layout.tsx or layout.js** file inside parent folder.
Note – Include children as a prop inside layout file



MetaData -
It helps to define metadata for each page. It is used to improve Seo of website

static Metadata => import MetaData api from next

```
export const metadata : Metadata  = {
    description : "This is provided by nextjs",
}
```

Dynamic Metadata => wrap the code inside “generateMetadata” function.

Navigating to different routes on any action -

We use <Link> Component that replaces <a> tag,

Syntax –
<Link href = “route_name”>Click Me</Link>

Eg -
<Link href = “/blog”>Click Me</Link>

To programmatically switch between routes -

We use useRouter provided from next/navigation.
Syntax - > useRouter.push(route_name)
Eg – useRouter.push(“/”)

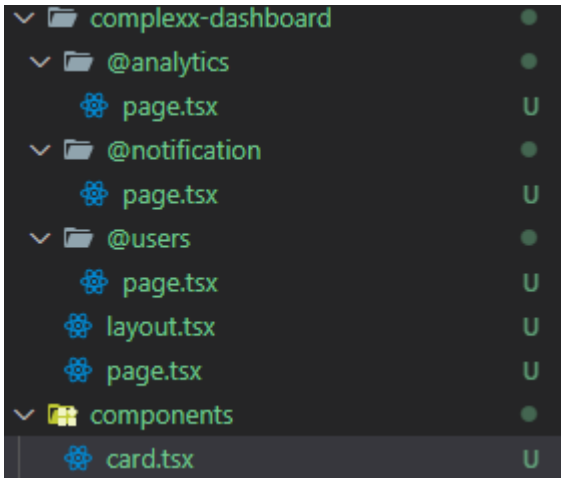
Loading file -
To create a pause between pages, loader file is created
Syntax – loader.tsx or loader.js

Error file -
To show error content, error file is created
Syntax – error.tsx or error.js

Global error file-
To show any error at the top level of hierarchy, we use **gobal-error.tsx or global-error.js** in the route

Parallel Route -
It is used when one layout route is divided into different slots.

Different slots folder should be named with convention=> @foldername



Create a separate component the wrap all the slots into one component

Import the component created and then enclose the slots within component tag

For eg -

```tsx
export const Card = ({children} : {children : React.ReactNode}) => {

    return (
        <div>{children}</div>
    )
}


export default Card
```

```tsx
import { Card } from "../../components/card";

const analyse = () => {

    return (
        <Card>Analysis Card</Card>
    )
}

export default analyse
```

Rendering –

Converting code component (source code) into interactive UI elements

Client Side Rendering -
When users makes request to a website, plain html and a reference to JS(powerhouse of source code) file is downloaded on clients machine
Now as soon as html is processed , JS file will be downloaded as soon as the html is processed

Server Side Rendering -
It's slightly different from CSR , in this type of rendering, the html is parsed on the server side and a non-interactive UI is presented to client along with reference to JS file.

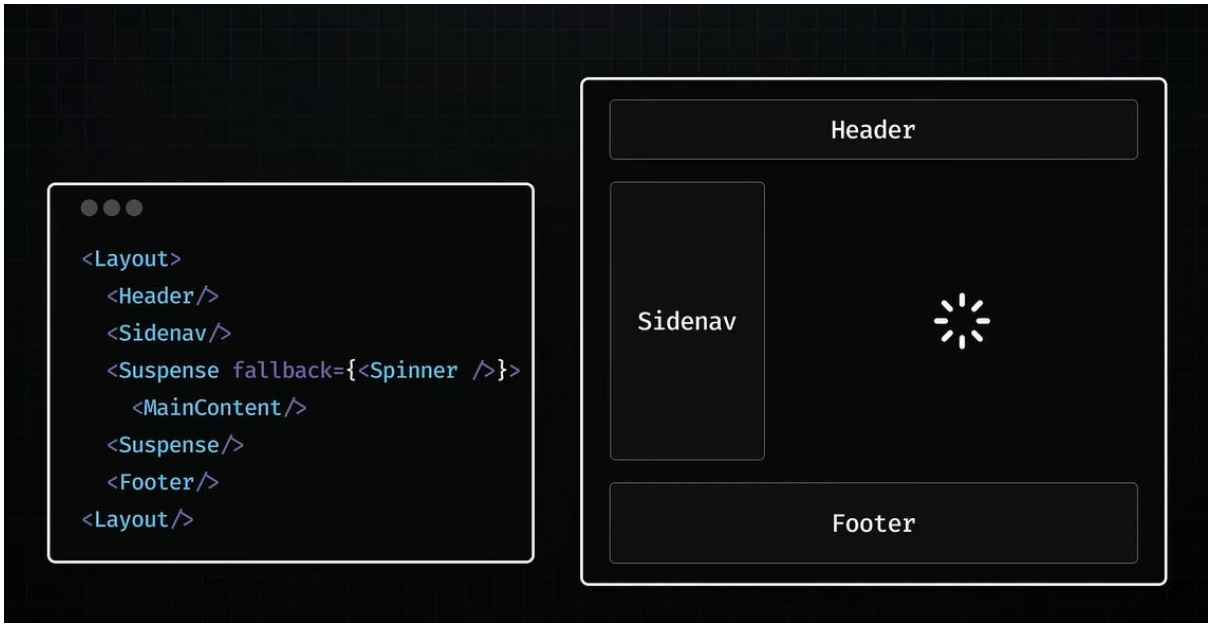
On client side, hydration happens to non-interactive UI to make it an interactive UI,

Hydration is the process of mapping all the html elements and hooking up it with JS logic

Static-Site Generation (SSG) –
This happens during build time as soon as the application is deployed on server. All the pages are rendered together.

Server-Side Rendering (SSR) -
It renders the pages on-demand when users request them.

To improve the problem of hydration of certain elements of page and get all the data needed for the page, we use <Suspense> component.



Using Dynamic keyword, we can do selective hydration -

It helps to load and hydrate components when needed. Doesn't render component on first build of application

```
const MyComponent = dynamic(() => import("./MyComponent"));
```

Note -

To expose the environment variable to browser, prefix the variable with - "NEXT_PUBLIC_(Variable_name)

For e.g. – NEXT_PUBLIC_API="www.themealdb.com/api/json/v1/1/search.php?s="

---

useSearchParams -

It is a next JS functionality to get the parameter values from URL. It can be used in both server and client-side components

Import the hook from "next/Navigation

Syntax -
useSearchParams.get("<name_of_parameter>")

Eg -
useSearchParams.get("id")

To load the image using Image -

Modify next.config.ts -

```
images: {
  remotePatterns: [
    {
      protocol: 'https',
      hostname: 'www.themealdb.com',
    },
  ],
},
```

Now use ,

```
<Image key={md.idMeal} src={md.strMealThumb} alt="Meal" width={300} height={300} />
```